

Review complet — GesAFPED + SIGA

Date : 2026-05-16 | Réalisé par : 5 agents Claude en parallèle (read-only) | Périmètre : backend Django/DRF `sigasiga` + frontend Next.js 16 `gesafped_frontend`
Type d'analyse : audit statique du code, aucune modification, aucune exécution destructive.

Synthèse exécutive

État global : Le projet est **plus maintenable que la moyenne d'un projet mono-dev** grâce au CLAUDE.md, aux conventions TanStack Query strictes côté frontend et à un découpage Django par apps cohérent. Cependant, **3 fuites de secrets dans git** exigent une action immédiate, et **plusieurs god-views backend ont continué à grossir** depuis la documentation officielle.

Top 3 actions urgentes :

- Purger `deploy/dump.sql` de l'historique git + rotation des mots de passe admin/IT/DE/scolarité.
- Branchement effectif de `captureError` (monitoring quasi-mort actuellement) pour gagner en visibilité prod.
- Refactor du N+1 sur `_compute_charge_permanents` (jusqu'à 900 requêtes par appel) + ajout des index manquants.

Score onboarding : 6.5 / 10 — passe à 8.5 / 10 avec 2-3 jours de polish (README racine, .env.example frontend, CI frontend).

Sommaire

- Sécurité (OWASP, secrets, RBAC, auth)
- Architecture (couplage, code mort, anti-patterns)
- Qualité de code (tests, complexité, lisibilité)
- Performance (N+1, index, bundle, re-renders)
- Maintenabilité (onboarding, doc, bus factor)
- Plan d'action priorisé
- Limites & vérifications honnêtes

1. Sécurité

1.1 Findings critiques **CRITIQUE**

1.1.1 Dump SQL avec hashes pbkdf2 de tous les utilisateurs committé dans git

Fichier : `sig deploy/dump.sql` (3063 lignes, commit `de8868b` du 08-05-2026)

Le dump contient les hashes `pbkdf2_sha256` de **tous les utilisateurs** (admin `sejad`, IT, étudiants, enseignants) avec username, email, role, NNI étudiants. Ces hashes sont crackables hors-ligne. De plus, plusieurs comptes étudiants/enseignants ont pour mot de passe leur NNI/nbac (cf. finding 1.2.1) — bypass possible si l'attaquant connaît un NNI public.

Action : `git filter-repo --path deploy/dump.sql --invert-paths`, rotation immédiate des mots de passe admin/IT/DE/scolarité, ajout au `.gitignore`.

1.1.2 Secrets de production sur disque dans deploy/.env

Fichier : `sig deploy/.env`

Le fichier contient `SECRET_KEY=a1IchqNNngwYTB9xZXzZBy16tnGOxJNFYU-...` et `DB_PASSWORD=siga_local_dev_2026`. Bien que `deploy/.gitignore` exclut `.env`, il faut vérifier qu'aucun commit historique ne le contient et qu'il n'est pas embarqué dans une image Docker poussée.

Action : `git log --all --diff-filter=A --name-only | grep deploy/.env` ; si trouvé → purge + régénération de `SECRET_KEY` et `DB_PASSWORD`.

1.1.3 .env racine du backend committé

Fichier : `sig/.env` (commit initial `a9c44041`)

Contient `SECRET_KEY=change-me-in-production`, `DEBUG=True`, `DB_PASSWORD=` (vide). Pas de secret réel exposé aujourd'hui, mais c'est le mauvais pattern : tout commit ultérieur qui injecte un vrai secret le rendra public. **Aucun `.gitignore` racine n'existe.**

Action : `git rm --cached .env` + créer `.gitignore` racine.

1.2 Findings hauts **HAUTE**

1.2.1 Mots de passe initiaux devinables (NNI/nbac)

Fichiers : `sig/apps/inscriptions/views.py:120` ; `apps/absence/views.py:431` ; `apps/prof/services.py:149` ;

`apps/authentication/management/commands/create_student_accounts.py:93`

Étudiants → `password=nbac` (numéro de bac, public). Enseignants → `password=NNI` (carte d'identité).
Le flag `doit_changer_mdp=True` est posé, mais tout compte non encore consommé est compromissible avec une information publique.

Recommandation : Générer un token aléatoire `secrets.token_urlsafe(12)` envoyé hors-bande (SMS/email/papier).

1.2.2 RBAC permissive par défaut sur les ViewSets sans `required_module`

Fichier : `sig/core/permissions.py:41-47`

`RBACPermission` ne s'applique que si `required_module` est défini sur la vue : `if not module_code: return True`. Tout `ViewSet` héritant uniquement de `IsAuthenticated` (default DRF) sans `required_module` est accessible à **n'importe quel utilisateur authentifié**.

Recommandation : Auditer chaque `ViewSet` pour exiger soit `required_module`, soit une `permission_classes` explicite. Ajouter un test smoke par module.

1.2.3 38 sérialiseurs utilisent `fields = '__all__'` (risque mass-assignment)

Exemple critique : `sig/apps/authentication/serializers.py:184` — `ModuleActionSerializer`.
Permet une édition non contrôlée via API (création/écriture de champs sensibles comme `created_by`, `is_active`, `validee_par`).

Recommandation : Remplacer `fields = '__all__'` par whitelist explicite + `read_only_fields`, au minimum sur les serializers RBAC (`Module`, `Action`, `ModuleAction`, `RoleDefault`, `UserPermission`).

1.2.4 CSRF absent sur les endpoints DRF cookie-auth

Fichier : `sig/core/authentication.py:11-30` + `sig/settings/base.py:60-73`

`CookieJWTAuthentication` identifie via cookie `httpOnly`, mais DRF bypass `CSRFViewMiddleware` quand l'auth n'est pas `SessionAuthentication`. Le frontend envoie déjà `X-CSRFToken` mais le backend ne le vérifie pas. Vecteur CSRF en POST cross-origin (cookie `SameSite=Lax` ne protège pas tous les cas).

Recommandation : Ajouter une vérification CSRF custom dans `CookieJWTAuthentication.authenticate`, ou passer `SameSite=Strict` en prod.

1.2.5 Régression CLAUDE.md règle non-négociable n°1 : 13 pages dashboard utilisent encore `apiFetch` direct

Devait être **0** selon CLAUDE.md. Pages concernées : `payement/{liste,ajouter,modifier/[id]}`, `emplois/gerer`, `enseignant/{reclamations,suivi}`, `inscriptions/derogations/ajouter`,

`parametres/semaines/ajouter` , `profs/{ajouter,[id]}` , `comptes/ajouter` , `em/ajouter` , `suivi/{rattrapage,remplissage}` .

1.3 Findings moyens MOYENNE

1.3.1 Endpoint public `/preinscriptions/suivi/?numero_dossier=<uuid>` (`inscriptions/views.py:163`) : `AllowAny` + throttle 20/min anonyme = 28 800/jour pour énumération distribuée. UUID = espace énorme donc risque acceptable, mais resserrer à 5/min.

1.3.2 `/institutions/active/` public (`apps/parametres/views.py:294`) : `InstitutionSerializer` avec `fields='__all__'` . Vérifier qu'aucun champ interne (paths absolus, tokens) n'est exposé. Créer un `InstitutionPublicSerializer` restreint.

1.3.3 `DEBUG=True` en `development.py` avec `ALLOWED_HOSTS=['*']` + `.env` racine committé avec `DEBUG=True` : risque info-leak si bascule prod oubliée.

1.4 Findings faibles FAIBLE

- **Swagger** `/api/docs/` **exposé sans auth** (`signa/urls.py:17`) : restreindre via `SERVE_PERMISSIONS = ['rest_framework.permissions.IsAdminUser']` .
- **Frontend stocke AuthUser en localStorage** (`lib/auth.ts:75`) : pas de tokens (cookies httpOnly OK), mais identité/rôle exfiltrables via XSS.
- **FileDropzone.tsx n'utilise pas validateUpload central** : risque de divergence entre les deux implémentations.
- **Pas de protection ReDoS sur lookup_value_regex** (`inscriptions/views.py:144`) : regex `[0-9a-f-]+` non bornée. Resserrer à un UUID strict.

Bonnes pratiques observées : JWT en cookies httpOnly, axes brute-force, throttling DRF, `safeNextUrl` / `safeImageSrc` / `validateUpload` en place côté frontend, pas de `dangerouslySetInnerHTML` , pas de tokens en localStorage, pas de `cursor.execute` avec f-string dans le code applicatif, pas de `pickle.Load` / `yaml.Load` , `password_hashers` Django par défaut.

2. Architecture

2.1 Top fichiers volumineux

Frontend

Fichier	Lignes	Note
app/dashboard/absences/stats/page.tsx	875	Combo data fetch + filtres + mutation seuil + charts
app/dashboard/evaluations/deliberations/[id]/page.tsx	816	Zone sensible (calculs notes)
app/dashboard/inscriptions/nouvelle/page.tsx	762	
app/dashboard/emplois/gerer/page.tsx	727	apiFetch direct restant
app/dashboard/absences/justificatifs/page.tsx	662	
components/scolarite/EtudiantDossier.tsx	621	
app/dashboard/absences/saisir/page.tsx	588	
app/dashboard/parametres/institutions/page.tsx	557	
app/dashboard/payement/liste/page.tsx	554	typo URL + apiFetch direct
app/dashboard/absences/rapport/page.tsx	550	

Backend — métriques actualisées vs CLAUDE.md

Fichier	Lignes (audit)	Lignes CLAUDE.md	Évolution
apps/evaluations/views.py	2592	2519	+73
apps/vacation/views.py	1840	1048	+792 (a doublé)
apps/avancement/views.py	1330	—	nouveau dans top 5
apps/suivi/views.py	1322	—	
apps/absence/views.py	1147	—	
apps/documents/services.py	1119	—	
apps/portail/views.py	996	—	

2.2 Code mort / dette

- **13 pages dashboard avec apiFetch direct** (régression CLAUDE.md règle #1, déjà listées en section 1.2.5).

- **41 routes orphelines** non liées depuis `lib/nav-config.ts`. Sous-routes `[id] / ajouter` légitimes, mais `dashboard/statistiques`, `dashboard/scolarite/documents`, `dashboard/parametres/{departements,salles}` sont suspectes.
- `components/charts/index.tsx` : barrel jamais importé — les pages utilisent l'import direct via Next.
- **Typo `payement` confirmée et étendue** : 9 fichiers concernés (au-delà des 6 documentés dans CLAUDE.md).

2.3 Anti-patterns récurrents

- **58 pages utilisent `useQuery` inline** sans passer par un `*-hooks.ts` factory, alors que 26 factories existent dans `lib/api/`.
- **20+ pages avec `useMutation` sans `invalidateQueries` dans `onSuccess`** → UI stale après écriture (bug utilisateur direct). Liste partielle : `absences/{etudiant,fiches,rapport,stats}`, `avancement/*`, `documents/{generer,registre}`, `enseignant/{premier-acces,vacations}`, `evaluations/{emargement,ponderation}`, `payement/{details,etat}`.
- **10+ listes paginées sans `placeholderData: keepPreviousData`** : toutes les `absences/*` et `avancement/*`.
- **30+ pages mixent `useState` + `useEffect` + `fetch`** au lieu d'utiliser TanStack Query pour le server state.
- **40+ fichiers redéclarent leurs propres interfaces** (`Departement`, `Filiere`, `Prof`, `Semestre`, `Salle`, `Etudiant`, `Vacation`). Aucun n'importe depuis `@/types/`.
- `absences/stats/page.tsx` = **combo des 4 anti-patterns** : `apiFetch` direct + `useQuery` inline + 4 interfaces locales + 875 lignes.
- **Couplage backend bidirectionnel `evaluations` ↔ `inscriptions`** : chacun importe l'autre via `apps.X`.
- `apps/avancement/views.py` **importe 7 apps** (`departement`, `em`, `emplois`, `parametres`, `prof`, `suivi`, `vacation`) — couche agrégat candidate à un module `services/`.
- `evaluations/views.py` : 29 `@action` + 5 surcharges `get_queryset` dans 2592 lignes.

3. Qualité de code

3.1 Tableau récapitulatif

Métrique	Backend (siga)	Frontend (gesafped)
Tests	6 fichiers <code>tests/test_*.py</code>	0 test
Config tests	<code>pytest.ini</code> + <code>conftest.py</code> + <code>factory_boy</code>	Aucune (pas de jest/vitest/playwright)
TODO/FIXME/HACK/XXX	0 dans <code>apps/</code>	0 dans <code>src</code>
any / @ts-ignore	n/a	9 <code>any</code> (4 fichiers, surtout <code>useRef<any></code> <code>chart.js</code>) — 0 <code>ts-ignore</code>
Fonctions > 100 lignes	~50+ (top : 1445 / 937 / 811 l)	196 fonctions > 100 l (top : 825 / 687 / 687)
Composants > 400 lignes	n/a	27 composants
<code>print()</code> en prod	0 hors migrations	n/a
<code>console.log</code> oubliés	n/a	0 (8 <code>console.warn</code> tous délibérés)
<code>except Exception:</code> nu	65 (22 fichiers)	n/a
<code>#006633</code> hardcodé	n/a	931 dans 190 fichiers (vs 877 documentés)
Linters configurés	Aucun (pas de black/ruff/pre-commit)	ESLint flat config — pas de prettier ni husky
<code>tsc --noEmit</code>	n/a	0 erreur (baseline préservée)
<code>captureError</code> usage	n/a	7 occurrences dans 2 fichiers seulement

3.2 Issues critiques

- 0 test frontend sur 131 pages dashboard.** Aucune protection sur RBAC (`lib/auth-roles.ts` , `lib/nav-filter.ts`), refresh JWT (`useTokenRefresh`), purge cache au logout (point critique CLAUDE.md), validation file, anti-XSS, anti open-redirect.
- 6 tests backend sur les modules les plus critiques** (calculs notes, délibérations, RBAC, vacation) — bon ciblage mais étroit. Pas de test sur `suivi` , `absence` , `portail` , `inscriptions` , `documents` , ni sur la Phase 5.
- 65 `except Exception: nus`**, concentrations : `portail/views.py` (12), `documents/services.py` (6), `authentication/views.py` (6), `vacation/views.py` (6), `suivi/views.py` (5). Catch-all masque les erreurs DB/validation/IO en 200/500 muet.
- Endpoints DRF retournant 200 sur condition d'erreur sémantique :** `absence/views.py:833` renvoie 200 avec `{'message': 'Aucun justificatif à supprimer.'}` (devrait être 404).

5. `captureError` **quasi-mort** : utilisé seulement dans `error.tsx` et `monitor.ts` . Les `catch (err)` dans les pages meurent silencieusement.
6. **931 #006633 dans 190 fichiers** (+54 vs CLAUDE.md). Pas de tokens, pas de variable CSS, pas de `:root` thématique. Tout rebrand demande un sed global.
7. **Aucun formater backend** : pas de `black` , `ruff` , `flake8` , `isort` , `pre-commit` , `setup.cfg` , `pyproject.toml` .

3.3 Top fichiers à complexité élevée (backend)

- `vacation/views.py:396` — bloc `_check_vacation_module` de 1445 lignes (faux positif partiel : englobe le reste du fichier).
- `absence/views.py:212` — 937 lignes
- `inscriptions/views.py:89` — `_creer_compte_etudiant` 811 lignes
- `evaluations/views.py:1368` — action `excel` de 500 lignes (à scinder)
- `evaluations/services/calcul_notes.py:33` — `_calculer_me_em` 577 lignes

4. Performance

4.1 Risques N+1 (top 5)

4.1.1 `_compute_charge_permanents` — IMPACT ÉLEVÉ

`apps/avancement/views.py:380-490` . Boucle `for prof in profs:` sur 150-300 enseignants avec, à chaque tour, 3 sous-requêtes (`SuiviePointage.objects.filter(prof_id=...)` , `Vacation.objects.filter(prof_id=...)` , `ChargeInstitution.objects.filter(prof=...)`). Jusqu'à **900 requêtes par appel**. Solution : 3 aggregate groupés `values('prof_id').annotate(...)` . Gain estimé : $\div 30-50\times$ la latence de l'endpoint.

4.1.2 `PresenceViewSet.bulk_update` — IMPACT ÉLEVÉ sur ressenti utilisateur

`apps/absence/views.py:562-582` . `update_or_create` dans `for item in d['presences']:` → 2 requêtes par étudiant. Une classe de 30 étudiants = 60 requêtes. Solution : `bulk_update` après préchargement, ou `INSERT ... ON DUPLICATE KEY UPDATE` MySQL natif.

4.1.3 `suivi/views.py:1024` : `{sp.pk: sp for sp in SuiviePointage.objects.filter(pk__in=ids)}` sans `select_related` → N+1 sur itération downstream.

4.1.4 `inscriptions/views.py:826` : action `elements` sans `prefetch_related('notes')` . Le `_notes_map` downstream peut repasser en N+1.

4.1.5 20 `for ... in Model.objects.all()` dans `evaluations/views.py` — à réviser cas par cas.

4.2 Index manquants suspectés

Table	Index manquants	Impact
<code>vacation_vacation</code>	Aucun <code>Meta.indexes</code> . Manque <code>(annee_univ, prof)</code> et <code>date</code>	ÉLEVÉ — table volumineuse
<code>suivi_suivie</code>	Seul <code>annee_universitaire</code> indexé. Manque composites avec <code>prof_id</code> et <code>em_id</code>	ÉLEVÉ
<code>suivi_suivie_pointage</code>	Aucun <code>Meta.indexes</code>	ÉLEVÉ (cible du N+1 #1)
<code>vacation_surveillance</code>	Pas d'index sur <code>(prof_id, annee_univ)</code> ni <code>date</code>	MOYEN

4.3 Frontend — usage TanStack Query & bundle

- **48% des `useQuery` sans `enabled:`** (91/176) — fetch précoce avant hydratation de `user.annee_universitaire`, puis re-fetch immédiat. Impact MOYEN.
- **Bundle sain** : pas de doublons date libs, charts lazy-loadés correctement (`components/charts/index.tsx`), `lucide-react` tree-shakable, `staleTime: 30s`, `refetchOnWindowFocus: false`, `gcTime: 5min` dans `lib/query-client.ts`.
- **11 `<img>` natifs** au lieu de `next/image` (photos étudiants notamment).
- **`useTimeout` peu utilisé** (3 fichiers seulement) — la plupart des `setInterval` sont manuels. Risque latent.

4.4 Backend — caching

- Pagination DRF activée globalement (`PAGE_SIZE=10`). RAS.
- **`cache.`** utilisé seulement dans 3 fichiers. Endpoints référentiels (filières, années, départements, jours, créneaux) servis sans cache HTTP/Redis. Gain facile via `@method_decorator(cache_page(60))`.

5. Maintainabilité

5.1 Score onboarding : 6.5 / 10

Un dev junior peut faire tourner les 2 stacks en quelques heures grâce aux Dockerfiles et au `.env.example` backend, mais doit deviner plusieurs choses critiques : frontend sans `.env.example`, ni README, ni script `typecheck` dans `package.json` malgré la règle d'or CLAUDE.md.

5.2 Documentation

Présent & utile

- `CLAUDE.md` frontend — excellent, à jour, conventions claires.
- `docs/TANSTACK_QUERY_GUIDE.md`, `docs/AUDIT.md`.
- `deploy/README.md` côté backend (vrai mode d'emploi prod).
- `sig/.env.example` documenté ligne-à-ligne.
- `drf-spectacular` expose `/api/docs/` et `/api/redoc/`.

Manquant

- **Aucun** `README.md` à la racine ni frontend ni backend.
- **Aucun** `.env.example` frontend.
- Pas de `CONTRIBUTING.md`, pas de `ARCHITECTURE.md` central, pas de schéma BD documenté hors migrations.

Bruit / dette doc

- Racine polluée par 5+ docs ad-hoc (`PLAN_FRONTEND_SCOLARITE.md`, `REMEDIATION_AUTH.md`, `collecte_anon.md`, `ipgei.md`, `portail_ens.md`, `releve.md`, `SUIVI_MODULE_CONTEXT.md` dupliqué dans `docs/`).
- Côté siga, `_diag*.py`, `_fix*.py`, `tmp_calcul.py`, fichiers de backfill JSON traînent à la racine repo.

5.3 Outillage dev

Bien

- Dockerfiles multi-stage front + back, `docker-compose.yml`, `deploy.sh`.
- Settings Django éclatés (`base/dev/prod/test/docker`).
- pytest avec markers + coverage gating en CI, `pytest --no-migrations`, `drf-spectacular`.

À combler

- `package.json` **n'expose pas de script** `typecheck` alors que `npx tsc --noEmit` est la "source de vérité" CLAUDE.md.
- **Aucun pre-commit hook** (ni husky côté front, ni `.pre-commit-config.yaml` côté back).
- **Pas de CI frontend** : seul `sig/.github/workflows/test.yml` existe.
- Aucun `.vscode/` ni `.idea/` partagés.
- Pas de `Makefile` côté siga.

5.4 Conventions de commit

Le CLAUDE.md mentionne le format `<jj-mm-aaaa>.<n>` (ex. `02-05-2026.3`). Sur les 100 derniers commits backend, **seuls 28/100 respectent ce format**. Le reste est libre (`fix bugs` , `eval++` , `vacs` , `before test` , `deploy siga`).

5.5 Bus factor estimé

- **Frontend : 1.5** — pattern TanStack ultra-cohérent, mais 931 `#006633` hardcodés, 40+ types redéclarés, typo `payement` jamais corrigée → une seule personne a la carte mentale complète.
- **Backend : 2** — settings/CI/test bien posés, mais 4 god-views totalisant 7000 lignes ne sont vraiment compris que par leur auteur. Migration Phase 5 et schéma `gesafped26` vs `gesafped` documentés uniquement dans la mémoire de l'agent.
- **Déploiement : 1** — `deploy/README.md` est bon mais sub-filter Nginx + placeholder `__SIGA_API_HOST__` = une seule personne sait déboguer si ça casse.

5.6 Refactorabilité — zones dangereuses (où il est risqué de toucher)

- **God-views** : `evaluations/views.py` 2592 l, `vacation/views.py` 1840 l, `avancement/views.py` 1330 l, `suivi/views.py` 1322 l.
- **God-pages frontend** : `absences/stats/page.tsx` 875 l, `evaluations/deliberations/[id]/page.tsx` 816 l.
- **Couverture inégale** : 6 fichiers tests pour ~705 fonctions backend. `vacation/views.py` , `suivi/views.py` , `documents/services.py` sont des boîtes noires.
- **Signaux Django magiques** dans `evaluations/signals.py` et `prof/signals.py` → non documentés dans CLAUDE.md.
- **Phase 5 (FK only)** : pas de test garde-fou, toute régression CharField restée invisible.
- **Ratio docstring backend ≈ 39%** (277/705 `def`). Acceptable mais fonctions critiques (calcul notes, délibération) parlent peu.

6. Plan d'action priorisé

#	Action	Effort	Impact	Quand
1	Purger <code>dump.sql</code> , <code>deploy/.env</code> , <code>.env</code> de l'historique git (filter-repo) + rotation des secrets exposés	0.5j	CRITIQUE	Aujourd'hui
2	Reset des mots de passe NNI/nbac actifs + génération token aléatoire pour comptes futurs	1j	HAUTE	Semaine 1
3	Audit RBAC : recenser tous les ViewSets sans <code>required_module</code> , ajouter perm explicite	1-2j	HAUTE	Semaine 1
4	Brancher Sentry (ou endpoint) + remonter <code>captureError</code> sur tous les <code>catch</code> non-triviaux	1j	HAUTE	Semaine 1
5	Migrer les 13 <code>apiFetch</code> directs restants vers les factory hooks existants	1-2j	MOYENNE	Semaine 2
6	Fix des 20+ <code>useMutation</code> sans <code>invalidateQueries</code>	1j	MOYENNE	Semaine 2
7	Refactor N+1 <code>_compute_charge_permanents</code> + 4 migrations d'index (vacation, suivi, surveillance)	2j	HAUTE perf	Sprint courant
8	Vectoriser <code>PresenceViewSet.bulk_update</code>	0.5j	MOYENNE perf	Sprint courant
9	Tests Vitest sur 6 modules sécu critiques (auth, RBAC, safe-redirect, safe-image, file-validation, force-logout)	1-2j	HAUTE	Sprint courant
10	Découper <code>evaluations/views.py</code> (2592 l) et <code>vacation/views.py</code> (1840 l) en sous-modules par <code>@action</code> group	3-5j	MOYENNE archi	Sprint dédié
11	Consolider <code>@/types/</code> + règle ESLint anti-redéclaration, migration des 40+ fichiers	2j	MOYENNE	Sprint dédié
12	Ajouter README racine + <code>.env.example</code> frontend + script <code>typecheck</code> + workflow CI frontend	0.5j	HAUTE onboarding	Semaine 2
13	Pre-commit hooks : <code>.pre-commit-config.yaml</code> backend (black/isort/ruff) + husky+lint-staged frontend	0.5j	MOYENNE	Semaine 2
14	Renommer <code>payement</code> → <code>paiement</code> (9 fichiers + nav-config + redirects Next)	1j	FAIBLE	Sprint dédié
15	Centraliser <code>#006633</code> en token CSS (préparation rebrand)	0.5j	FAIBLE	Backlog
16	Restreindre Swagger <code>/api/docs/</code> aux admins en prod	0.2j	FAIBLE	Backlog
17	Refactor 27 god-pages frontend >400 lignes	5j+	MOYENNE	Backlog supervisé

7. Limites & vérifications honnêtes

- Audit réalisé par **lecture statique uniquement**. Aucun runtime profiling, aucune exécution de tests, aucune migration appliquée.
- Les findings "N+1" et "index manquants" sont des **suspensions fortes** basées sur le code — à confirmer avec `EXPLAIN` MySQL ou `django-silk` en environnement de staging.
- Les CVE potentielles (Pillow 10.4.0, Django 4.2.16) ne sont pas vérifiées — lancer `pip-audit` avant priorisation.
- Les 3 findings CRITIQUE git nécessitent une vérification rapide :

```
git log --all --full-history -- deploy/dump.sql deploy/.env .env
```

pour confirmer l'ampleur exacte de la fuite.

- Audit produit par 5 agents Claude indépendants (sécurité, architecture, qualité, performance, maintenabilité) en parallèle. Le rapport consolide leurs sorties, déduplique et croise les constats.

Méthodologie : Chaque agent a accès en lecture seule aux deux working directories (`c:\react_projects\GES\sig` et `c:\react_projects\GES\gesafped_frontend`). Aucun fichier source n'a été modifié, aucune commande mutante n'a été exécutée. Les findings sont reproductibles via `grep` / `find` à partir des chemins listés.

Généré le 2026-05-16 par Claude Code (Opus 4.7).